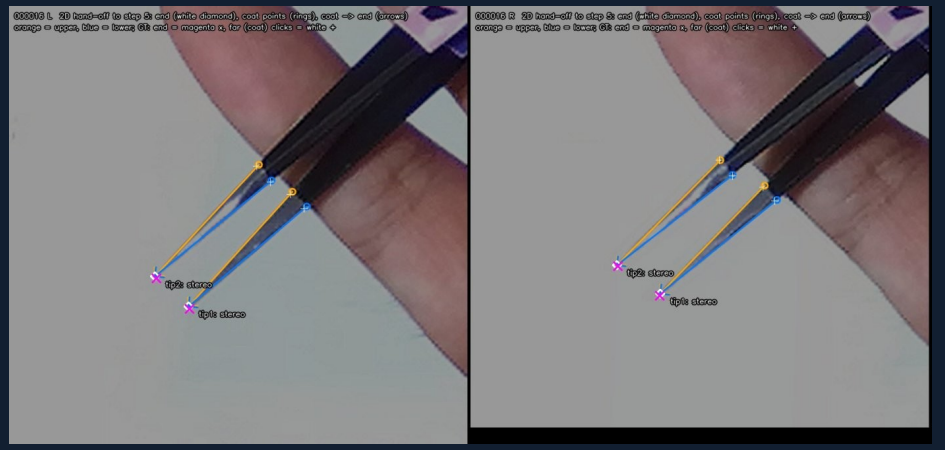
SEMANTIC TIP2D

스테레오 영상 기반 핀셋 팁 Semantic Keypoint 측정

VETUS · Pink tweezer | 구현 개요 , 알고리즘 로직 , 속도와 평가 결과



핵심 : pose 는 초기 위치만 제공하고 , 최종 keypoint 는 영상 edge· 밝기 ·stereo correspondence 로 다시 측정

한 장으로 보는 구현 결과

교수님께 먼저 전달할 핵심 메시지

무엇을 구현했나

3DGS semantic 정의를 실제 L/R 영상의 팁 끝점 · 코팅 경계 · 방향으로 변환하는 영상 기반 측정 알고리즘

왜 필요한가

6D pose 는 2D 초기 위치에는 유용하지만 깊이 오차가 큼 . pose 를 그대로 쓰지 않고 영상과 stereo 로 재측정

현재 출력

현재 semantic-tip2d: L/R 2D 끝점 , upper/lower 코팅점 , 윤곽 직선 , 코팅→끝 방향

47 ms
VETUS / GPU 정렬

36 ms
Pink / GPU 정렬

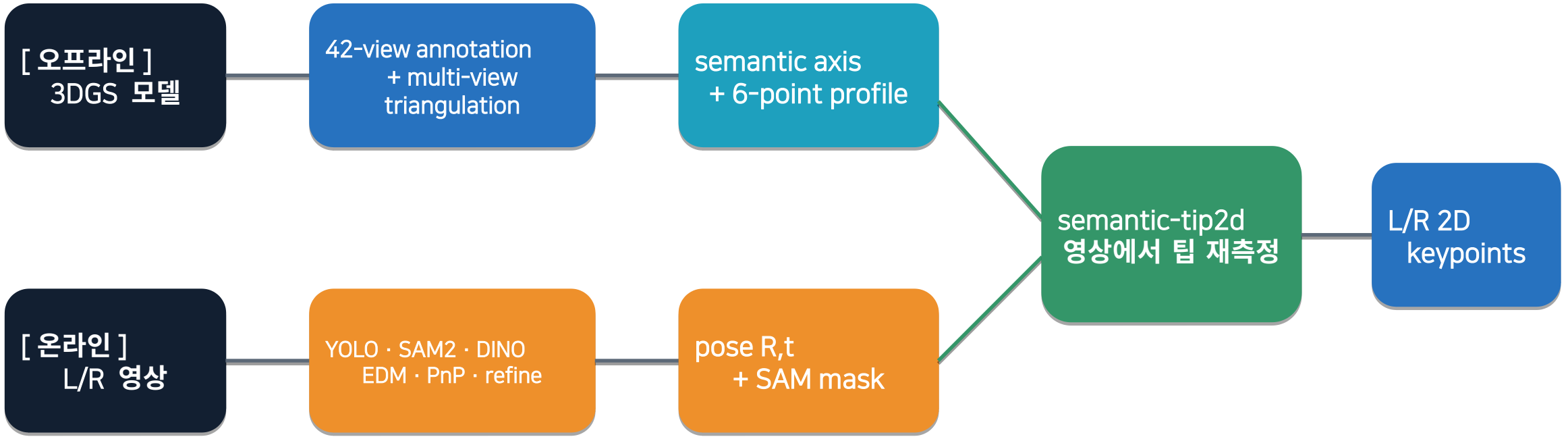
0.97 px
VETUS endpoint-GT 중앙값

0.81 px
Pink 정상 pose endpoint-GT

GT: 사람이 L/R 영상에 클릭한 팁 끝점 · 오차 : 검출 endpoint 와 클릭 GT 사이의 2D pixel distance

전체 시스템에서의 위치

오프라인 semantic 정의 + 온라인 pose + 팁 정밀 측정



pose 는 '어디쯤 / 어느 방향'인지 알려주는 prior 이며 , 최종 위치는 영상에서 다시 잰다 .

근거 : README.md, semantic_keypoint_generation/docs/PIPELINE_MANUAL_KO.md

제가 구현한 범위

물체 등록 도구부터 스테레오 팁 측정 및 검증까지

A. Semantic 정의 생성

3DGS 42-view 렌더
base/end 다중 시점 annotation
RANSAC 삼각측량과 reprojection 검증

- 결과 : 팁 6 점 profile 생성

B. 영상 기반 팁 측정

GS alpha silhouette 생성

- 방향성 chamfer 정렬

gradient/robust line 기반 윤곽 재검출 Canny chain + ridge 끝점 검출

- 밝기 기반 코팅 경계 검출

C. Stereo· 최적화 · 평가

- 동일 행 조건으로 L/R 대응 선택
- 수평 팁 길이 규칙

CPU/GPU 동일 결과 검증
crop alpha 렌더 최적화
GT 클릭 기반 2D pixel 오차 평가

현재 패키지의 입력과 출력

진입점 : tip2d.pipeline.detect_tips_2d

입력

L/R query

1920×1080 rectified stereo BGR 영상

L/R SAM mask

물체 주변 edge 를 제한하는 binary mask

pose R, t

6D pose estimation 출력
R: 왼쪽 rectified 카메라에서 본 물체 회전
t: 같은 시점의 물체 위치

object + renderer

semantic axis, profile, 3DGS alpha renderer

detect_tips_2d(...) → tip1 / tip2

출력

end_px

L/R 팁 끝 픽셀

edges

upper/lower 코팅점과 윤곽 직선

coat_end_dir

코팅→끝 2D 단위벡터

quality flags

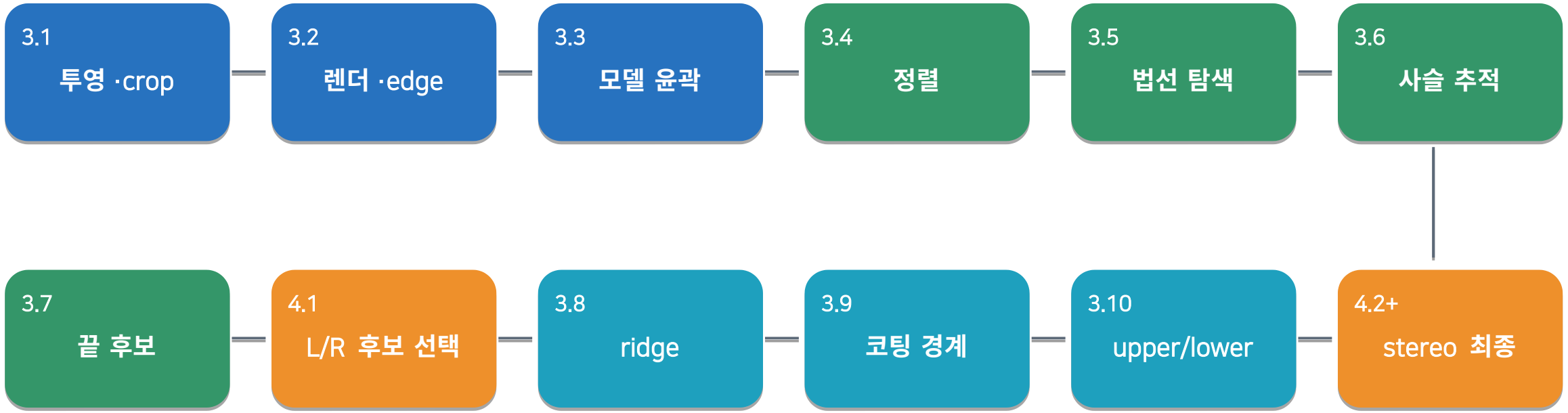
행 차이 · 경계 · 수평 · 정렬 한계 · 실패

코드 : tip2d/pipeline.py

실제 실행 순서

문서 번호와 호출 순서가 다른 지점 : 4.1 이 3.8 보다 먼저 실행

L/R 각각

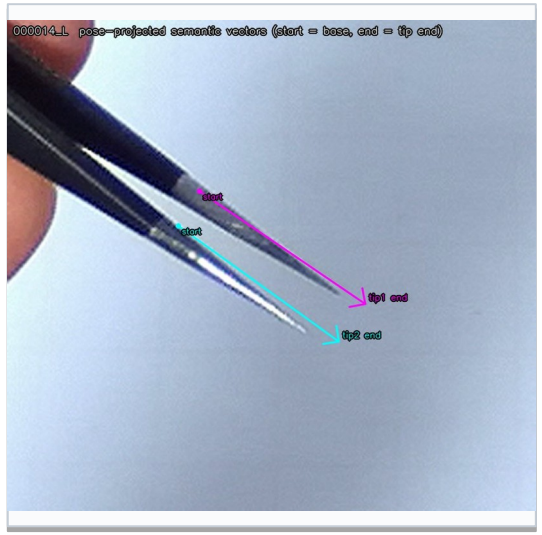


L/R 공동 선택 → 다시 view 마무리 → 공동 선택

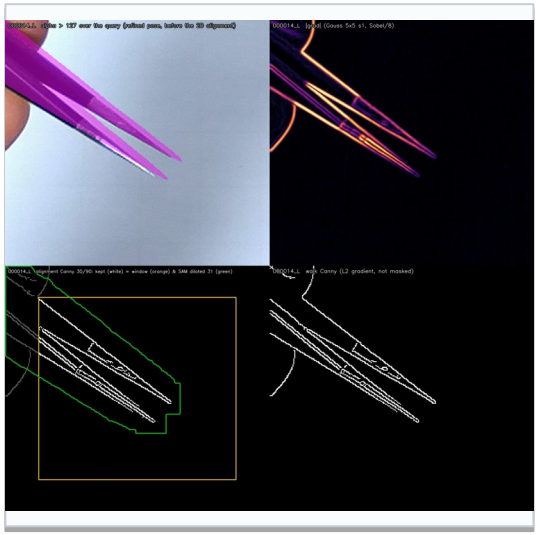
근거 : outputs/VETUStweezers/debug/README_KO.md

3.1~3.4 | Pose 투영을 실제 영상 edge 에 정렬

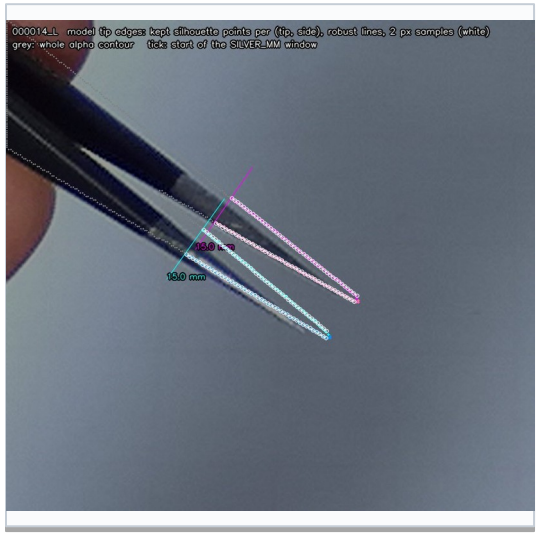
이 단계가 pose 의 2D 투영 오차를 흡수



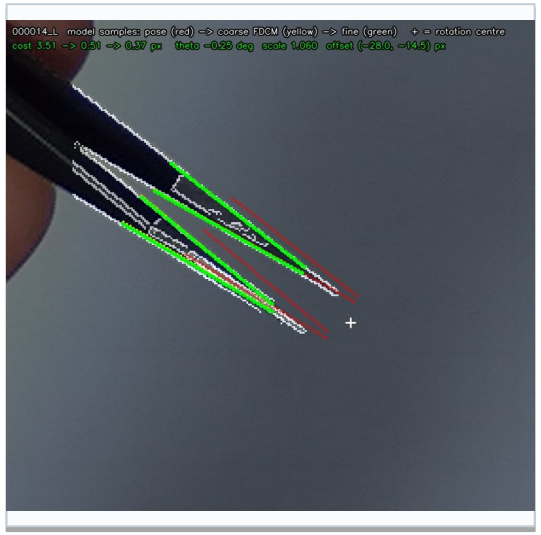
semantic vector 투영



alpha·Canny· 방향 거리 지도



팁 윤곽 직선 4 개



닮음 변환 최적화

닮음 변환이란 ?

형상을 찌그러뜨리지 않고 ① 회전 θ ② 모든 방향에 같은 배율 s ③ 평행이동 offset 만 적용 . 직선 · 각도 · 길이 비율은 유지

정렬식 · 탐색

$p' = \text{center} + s \cdot R(\theta) \cdot (p - \text{center}) + \text{offset}$
θ ±6°, s 0.96/1/1.04, offset ±40px → 0.25°/0.5px 세밀화

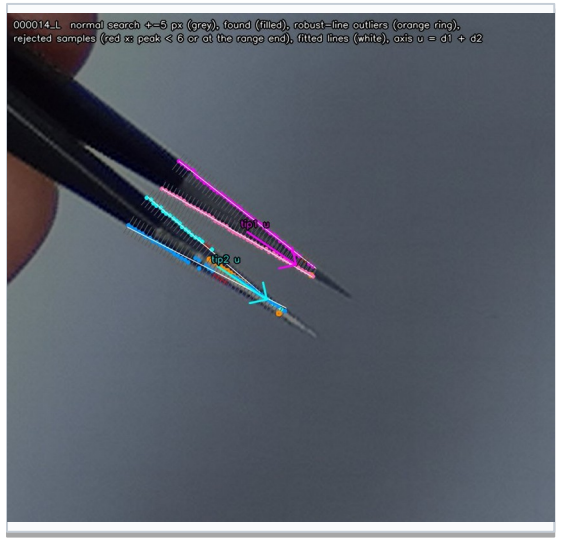
무엇을 보정하나

θ: 자세 각도
s: 투영 크기
offset: 2D 위치
VETUS 최대 이동 28px

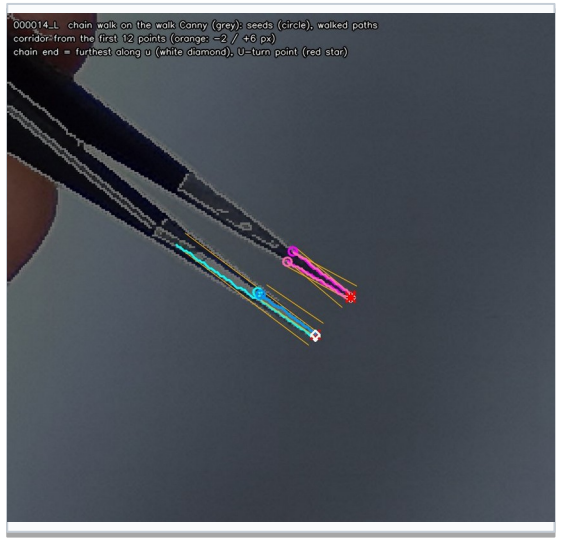
그림 : VETUS debug/view 3.1~3.4 에서 선별

3.5~3.8 | 실제 윤곽과 팁 끝 재측정

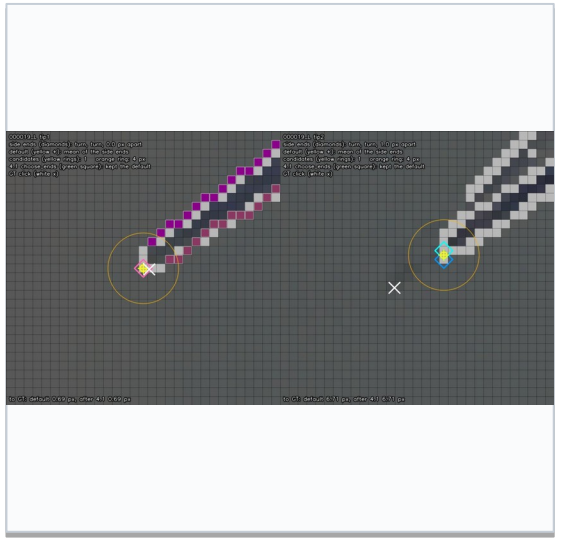
정렬된 모델선은 탐색 가이드일 뿐 , 측정값은 query 영상에서 생성



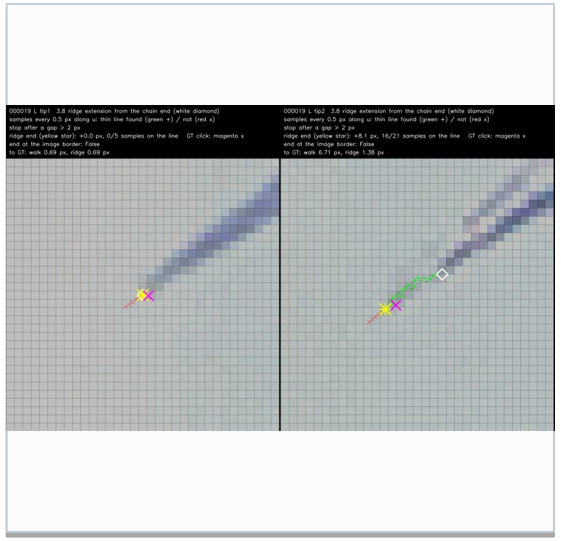
모델 직선의 법선 방향에서 실제 영상 edge 재탐색



Canny 8-neighbor least-turn walk + corridor



양쪽 chain 끝 평균 / U-turn 후보 생성



가는 ridge 를 최대 20px 연장 ; 조건부 채택

법선 탐색 로직 각 모델 샘플에서 단위 법선 n 방향 ±5px 탐색 → 영상 밝기 gradient g 와 법선의 내적 |g·n| 이 최대인 위치를 실제 윤곽점으로 채택

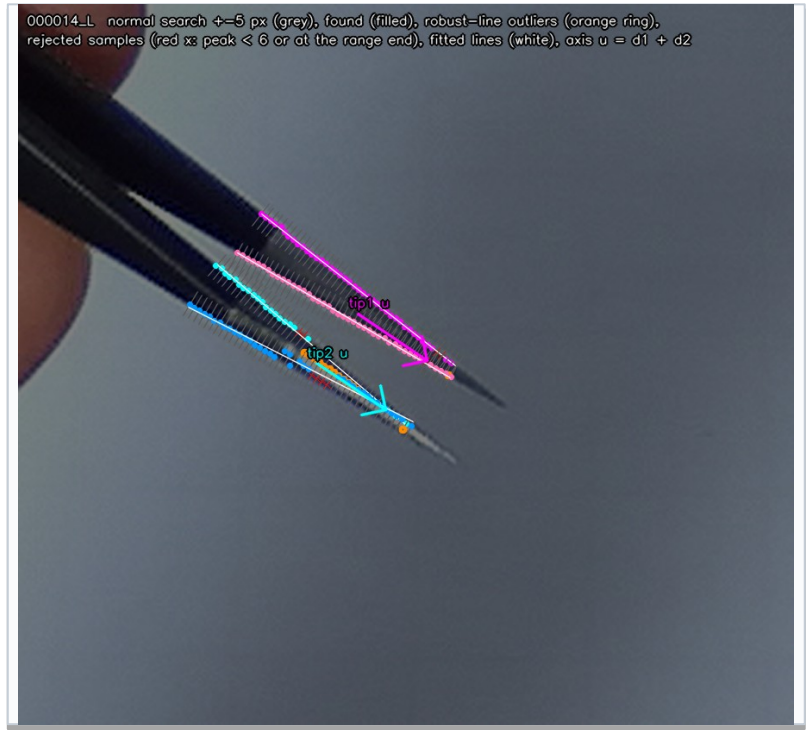
score ≥ 6, 탐색창 끝 제외 · 채택점 ≥ 샘플의 30%(최소 5 개) → Huber line fit 후 MAD 이상점 제거

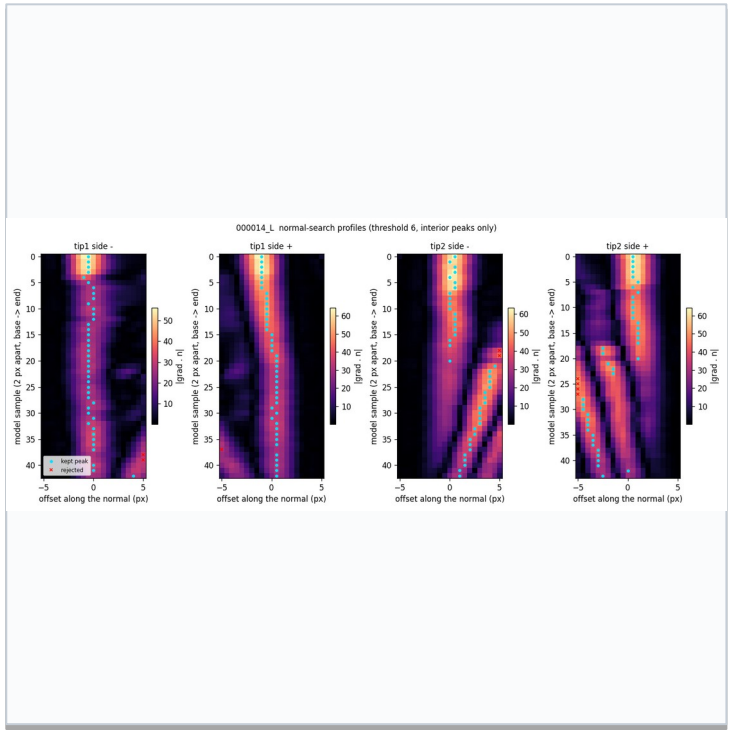
000019 L tip2

Canny 6.71px 오차
→ ridge 후 1.38px

3.5 | 법선 방향 edge 재탐색

정렬된 모델 윤곽을 시작점으로 사용하고 , query 영상의 gradient 에서 실제 edge 위치를 다시 찾는다





① 탐색선 생성

모델 윤곽을 2px 간격으로 샘플링 . 각 점 p_i 에서 직선 방향 d 에 수직인 $n = (-d_Y, d_X)$ 계산

② 영상 edge score

$g=(g_X,g_Y)$: 후보 픽셀의 Sobel 밝기 gradient
$n=(n_X,n_Y)$: 모델 윤곽의 단위 법선
$score=|g\cdot n|=|g_Xn_X+g_Yn_Y|$

③ 선택과 유효점 판정

법선 방향 밝기 변화가 가장 큰 score 최대점을 edge 로 선택 . 절댓값은 밝→어둠 / 어둠→밝음 모두 허용 . score ≥ 6, 창 끝 제외

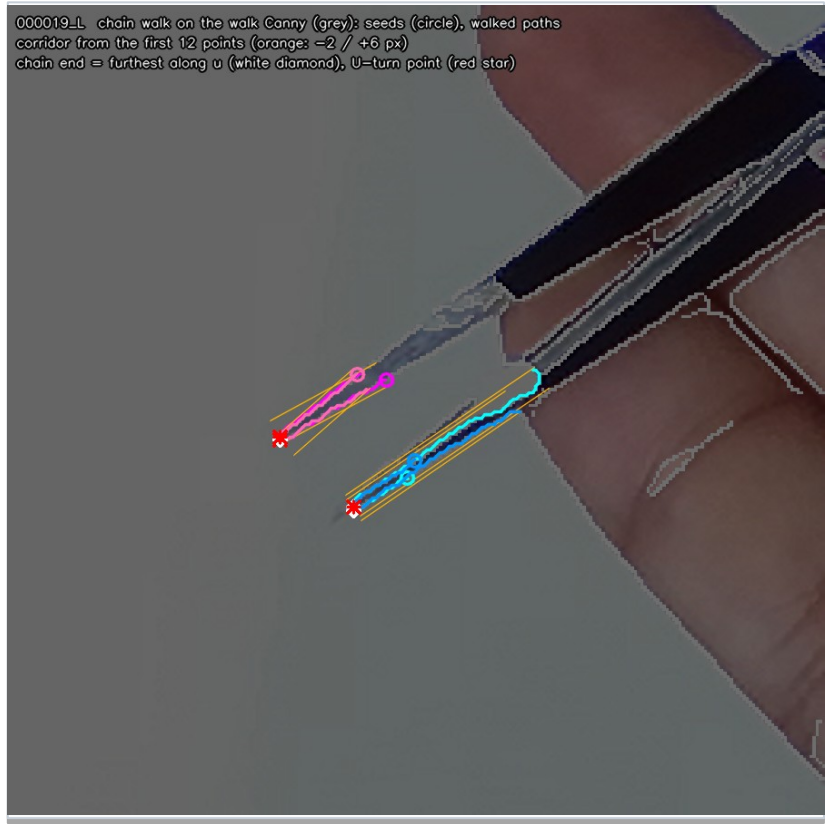
④ 유효점 ≥ 샘플의 30%(최소 5 개) → Huber line fitting → MAD 이상점 제거 → 팁 축 $u = unit(d_1+d_2)$

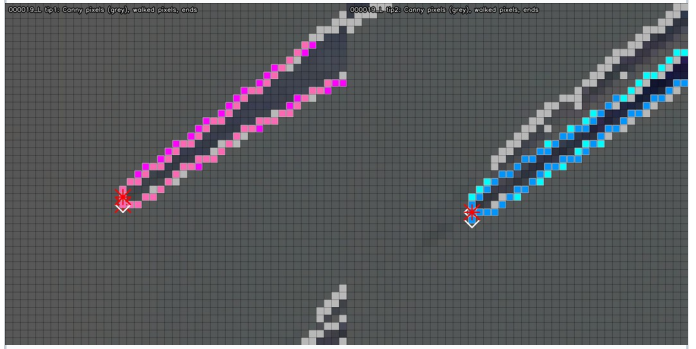
※ MAD (Median Absolute Deviation): 잔차의 중앙값으로부터 절대편차의 중앙값 . 큰 잔차를 가진 윤곽점을 이상점으로 판별하는 robust 통계량 .

왼쪽 : 법선 탐색 구간과 채택점 · 가운데 : 샘플별 gradient profile

3.6 | Canny 사슬 추적

윤곽선 위 seed 에서 시작해 가장 덜 꺾이는 Canny 이웃을 따라 팁 끝까지 이동





① Seed

윤곽선에서 팁 방향으로 가장 먼 점보다 15px 뒤에서 시작 . 주변 7×7 Canny 중 가장 가까운 픽셀 선택

② Least-turn walk

8-neighbor 중 현재 진행 방향과 cosine 이 가장 큰 , 즉 가장 덜 꺾이는 미방문 edge 픽셀로 이동

③ Self corridor

처음 12 점으로 직선을 맞춘 뒤 그 선 기준 바깥 2px, 물체 안쪽 6px 복도 안에서만 추적

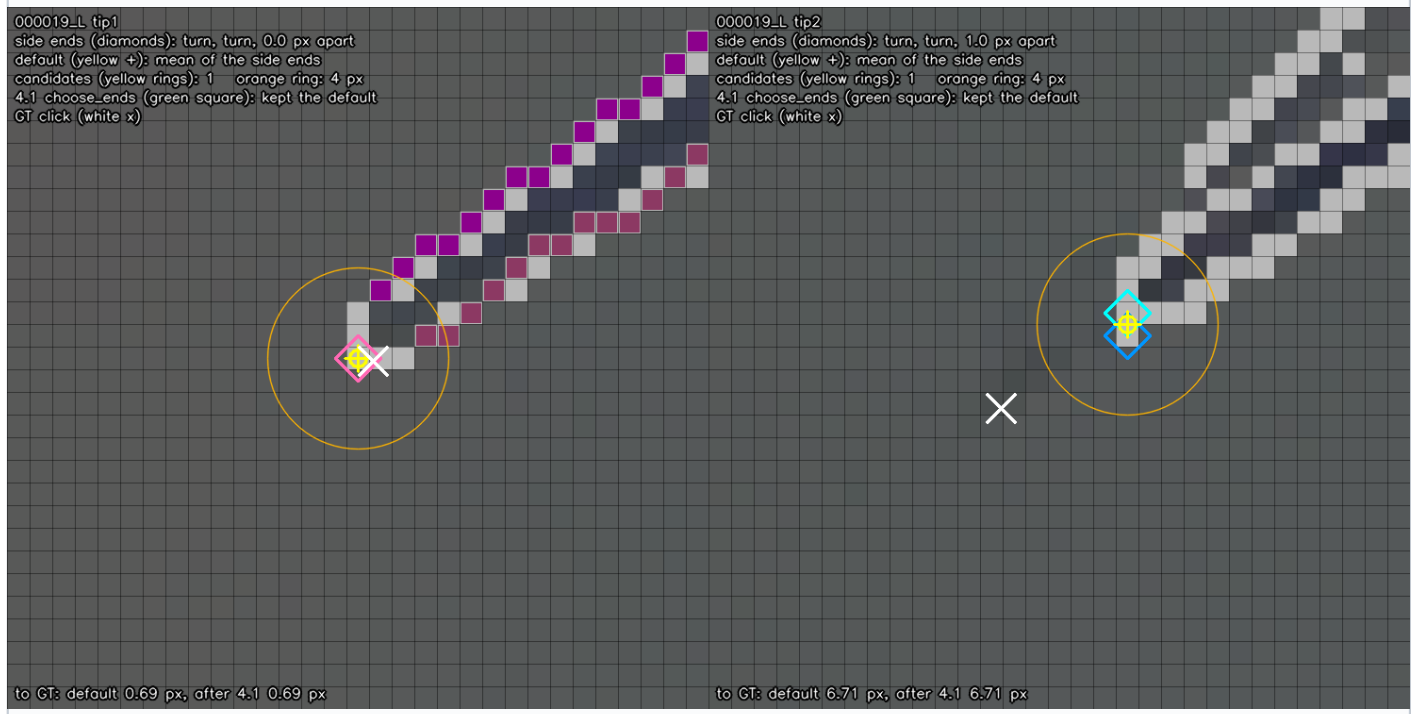
④ 종료 · 끝점

갈 이웃이 없거나 최대 step 도달 시 종료 . path·u 가 최대인 점이 chain end. >90° 회전 후 ≥ 3px 후퇴하면 U-turn

복도는 다른 물체 edge 로 새는 것을 막고 , U-turn 은 팁 끝을 돌아간 사슬을 판별

3.7 | 팁 끝점 후보 생성

양쪽 윤곽 사슬의 끝과 U-turn 정보를 조합해 기본점과 stereo 선택용 후보를 만든다



Case A · 두 끝이 가까움

두 side chain end 사이 거리 ≤ 4px
→ 두 점의 평균을 기본 endpoint 로 사용

Case B · 한쪽만 U-turn

두 끝의 거리 > 4px 이고 한 사슬만 U-turn
→ 팁 끝을 돌아간 U-turn 쪽 endpoint 채택

Case C · 모호한 경우

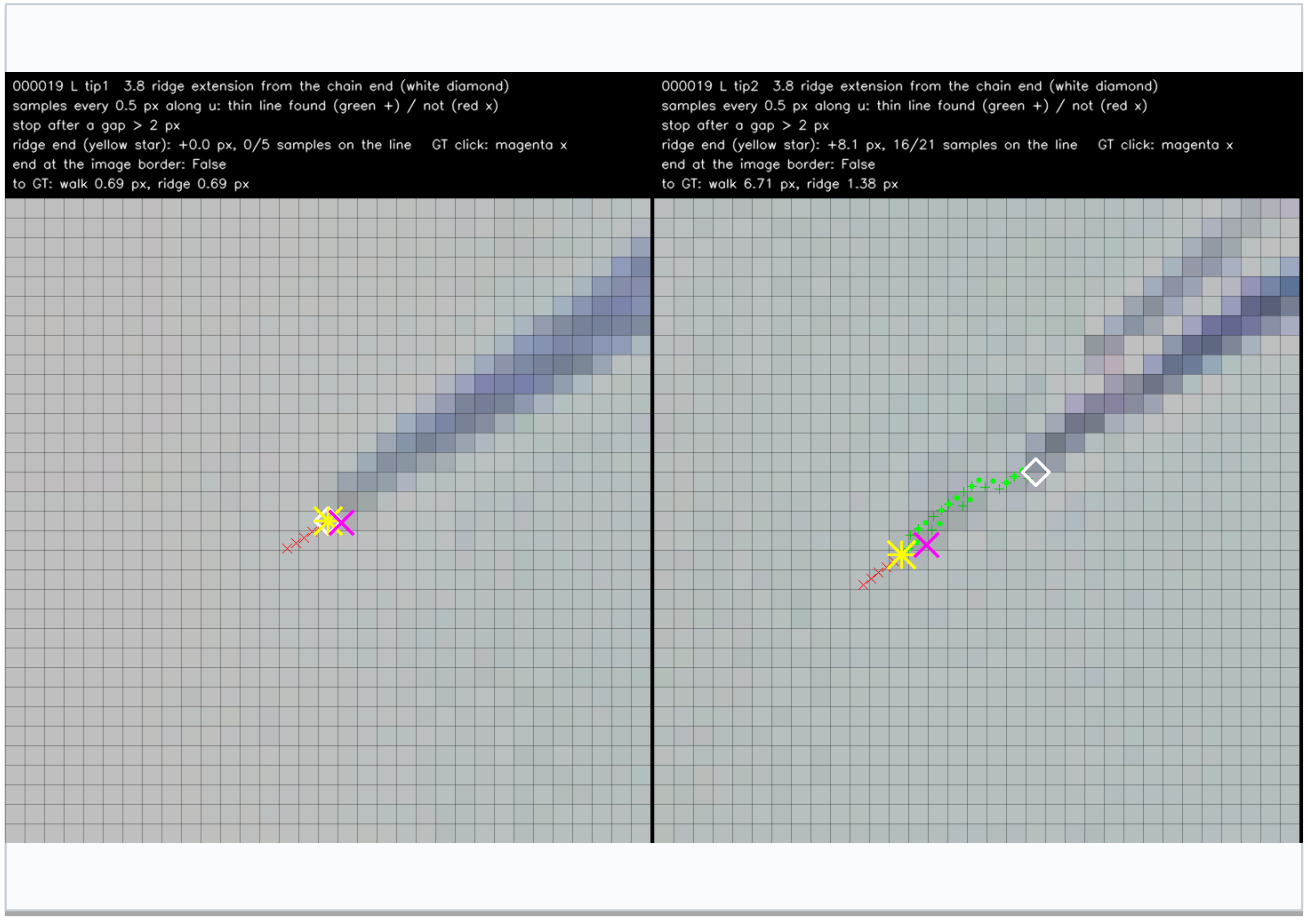
U-turn 판정이 같으면 두 끝의 평균을 기본값으로 두고, 각 side end 도 별도 후보 목록에 보존

다음 4.1 에서 L/R 행 차이가 가장 작은 후보 쌍을 선택

노란 +: 기본 endpoint · 노란 원 : 추가 후보 · 흰 x: 클릭 GT

3.8 | Ridge 연장

Canny 가 놓친 가늘고 흐린 팁 끝을 축 방향의 국소 밝기 ridge 로 보완



① 전진 sampling

선택된 chain end 에서 팁 축 u 방향으로 0.5px 씩 최대 20px 전진

② 횡단 profile

각 위치에서 축의 수직 방향으로 팁 폭 profile 을 읽고 , 양옆 background 의 median 과 비교

③ Ridge 판정

중앙 profile 과 flank 차이 ≥ 6 gray, 양쪽 flank 차이 < 6 일 때 가는 선이 이어진 것으로 판정

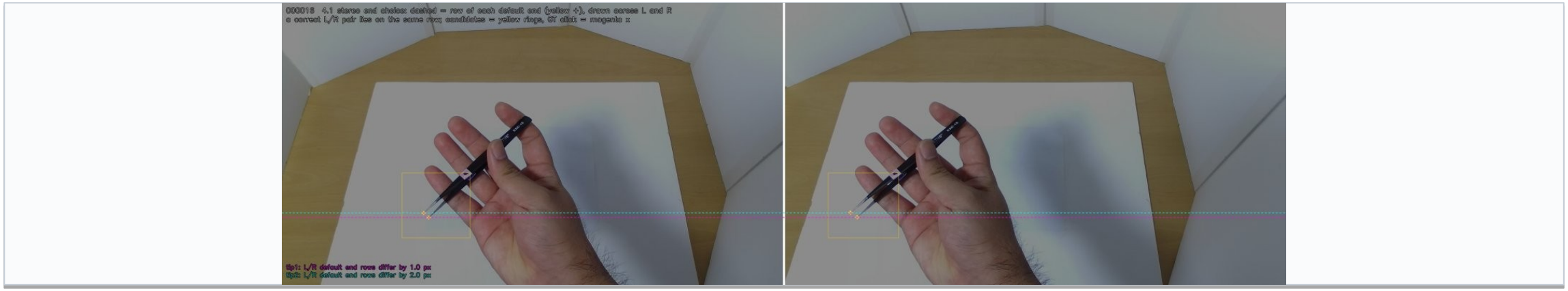
④ 조건부 사용

마지막 유효점을 ridge end 로 저장 . 2px gap 은 허용하되 , 항상 쓰지 않고 4.2 의 L/R 행 일치가 좋아질 때만 채택

000019 L tip2: walk-GT 6.71px → ridge-GT 1.38px

4.1~4.4 | Rectified stereo 제약으로 후보를 선택

실제 대응점은 왼쪽과 오른쪽 영상에서 같은 행에 존재



4.1 끝 후보

L×R 후보 중 행 차이가 최소인 쌍 . 기본보다 ≥ 1px 개선되고 ≤ 3px 일 때 교체

4.2 walk / ridge

두 종류의 끝점을 조합하여 L/R 행 차이가 더 작은 쌍만 채택

4.3 코팅 행 검사

행 차이 >1.5px 이면 바깥 밝기 경계 후보로 이동

4.4 수평 팁

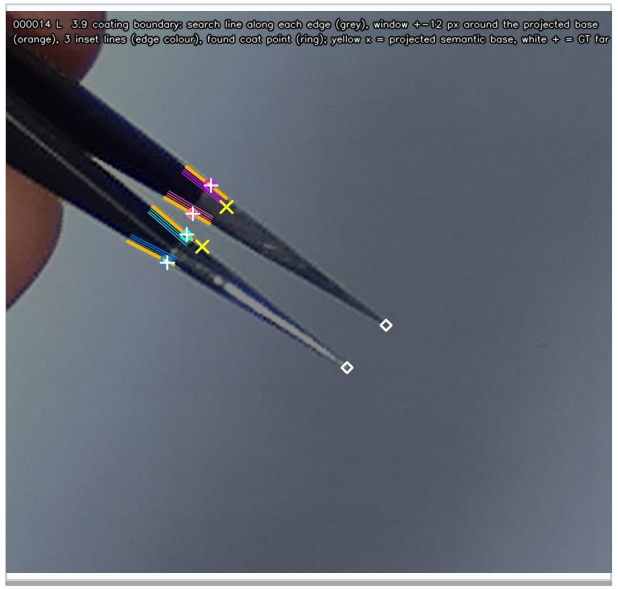
행 차이를 쓸 수 없으므로 stereo 후보 길이가 실제 팁 길이에 가까운 조합 선택 (선택에만 사용)

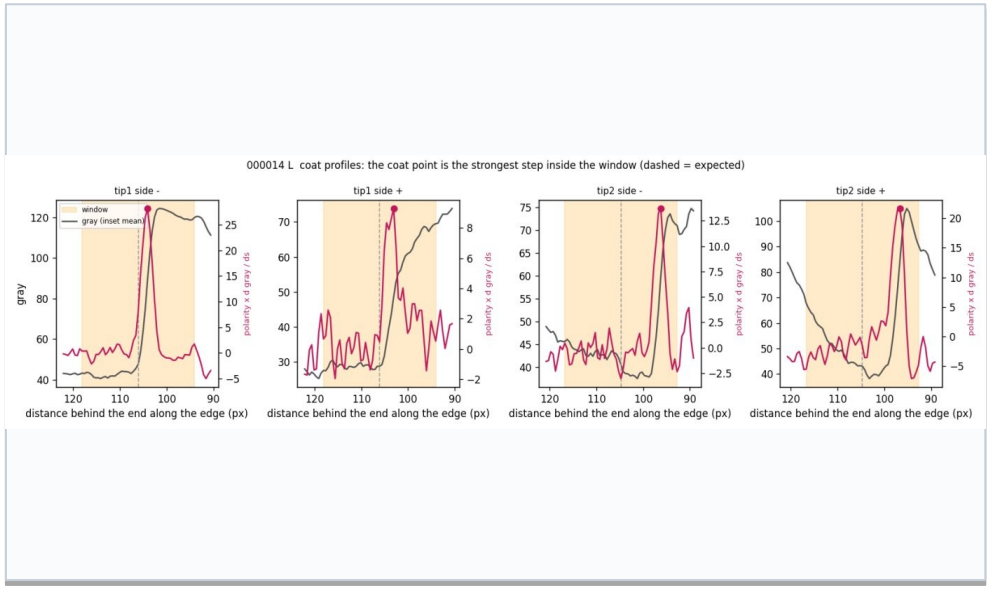
VETUS: 4.2 에서 3 개 팁 교체 · 4.3 에서 6 곳 중 4 곳 보정 · 수평 프레임 000018 은 ridge–ridge 선택

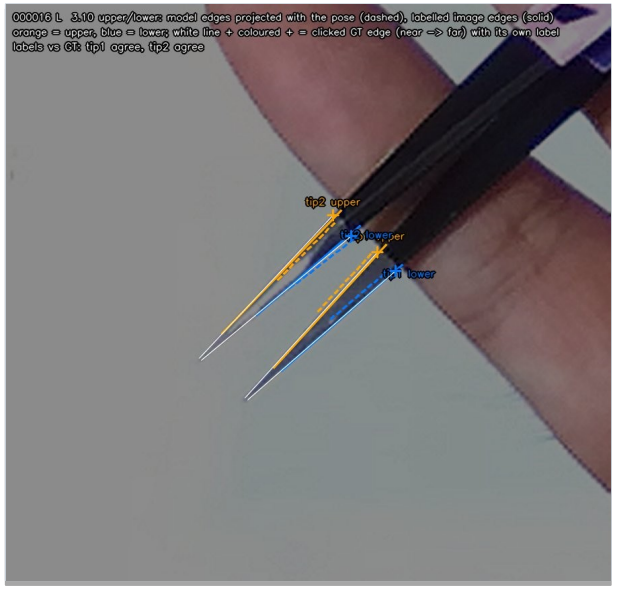
근거 : debug/pair/README_KO.md

3.9~3.10 | 코팅 경계와 의미 라벨

각 윤곽선에서 밝기 변화가 가장 큰 지점을 코팅-금속 경계의 2D keypoint 로 측정







① 기대 위치

검출 끝점에서 투영 semantic vector 길이만큼 뒤로 이동

② 밝기 profile

윤곽을 따라 ±12px, 안쪽 1.5/2.5/3.5px 세 줄 평균

③ 경계 선택

물체별 부호를 적용한 밝기 미분의 최대점 + 포물선 sub-pixel

④ 의미 라벨

6 점 profile 투영으로 upper/lower 결정 ; VETUS GT 27/27

코드 구조와 호출 관계

25 줄의 orchestration 과 역할별 모듈 분리

역할별 모듈 호출

pipeline.py
detect_tips_2d

```
for side in ("L", "R"):
    vectors =
project_vectors(...)
    alpha = render(...)
    states[side] =
view_candidates(...)
chosen =
choose_ends(states)
res = finish_view(...)
return pair_2d(res,
query, obj)
```

camera.py

카메라 모델 · 투영

gs_render.py

crop alpha 렌더

view.py

3.1~3.10 단일 view

alignment.py
align_gpu.py

3.4 CPU/GPU 정렬

stereo.py

4.1~4.4 L/R 선택

objects.py

물체 profile· 상수

실험 저장소의 약 4,100 줄 /15 개 폴더에서 실제 실행 경로만 패키지로 정리

하나의 알고리즘 , 물체별 profile 로 일반화

VETUS 에서 정한 px 상수를 6 점 profile 의 물리 크기 비율로 변환

항목	VETUS	Pink
형상	곧은 팁	짧고 꺾인 팁
길이 비율 sL	1.000	0.573
폭 비율 sW	1.000	0.672
모델 윤곽 창	15.0 mm	8.6 mm
코팅 밝기 방향	+1: 어두움→밝음	−1: 밝음→어두움
sub-pixel clip	없음	±0.5 px
정상 pose 출력	14/14 tips	10/10 tips

길이 관련 상수

REF × sL
seed, chain 길이 , ridge 범위 , coat window

폭 관련 상수

REF × sW
윤곽 복도 , 코팅 inset, ridge half-width

설정 파일만 교체 :
configs/objects/<name>.json

코드 : tip2d/objects.py, configs/objects/*.json

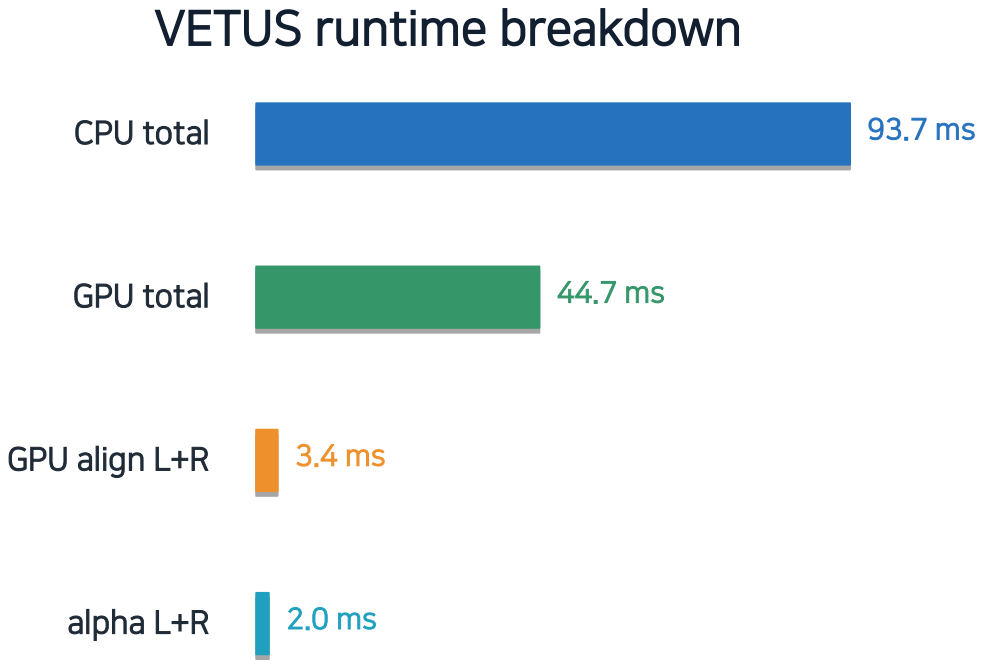
처리 속도

스테레오 한 쌍 , alpha 렌더 포함 중앙값 ; pose/SAM 및 이미지 로딩 제외

물체	CPU 정렬	GPU 정렬	GPU 처리율
VETUS	95.9 ms	47.2 ms	21 FPS
Pink	74.4 ms	36.1 ms	28 FPS

GPU 정렬 효과 (VETUS 상세 벤치마크)

정렬 view 당 약 25ms → 약 1.7ms
팁 단계 전체 93.7ms → 44.7ms
CPU/GPU 최종 결과 동일

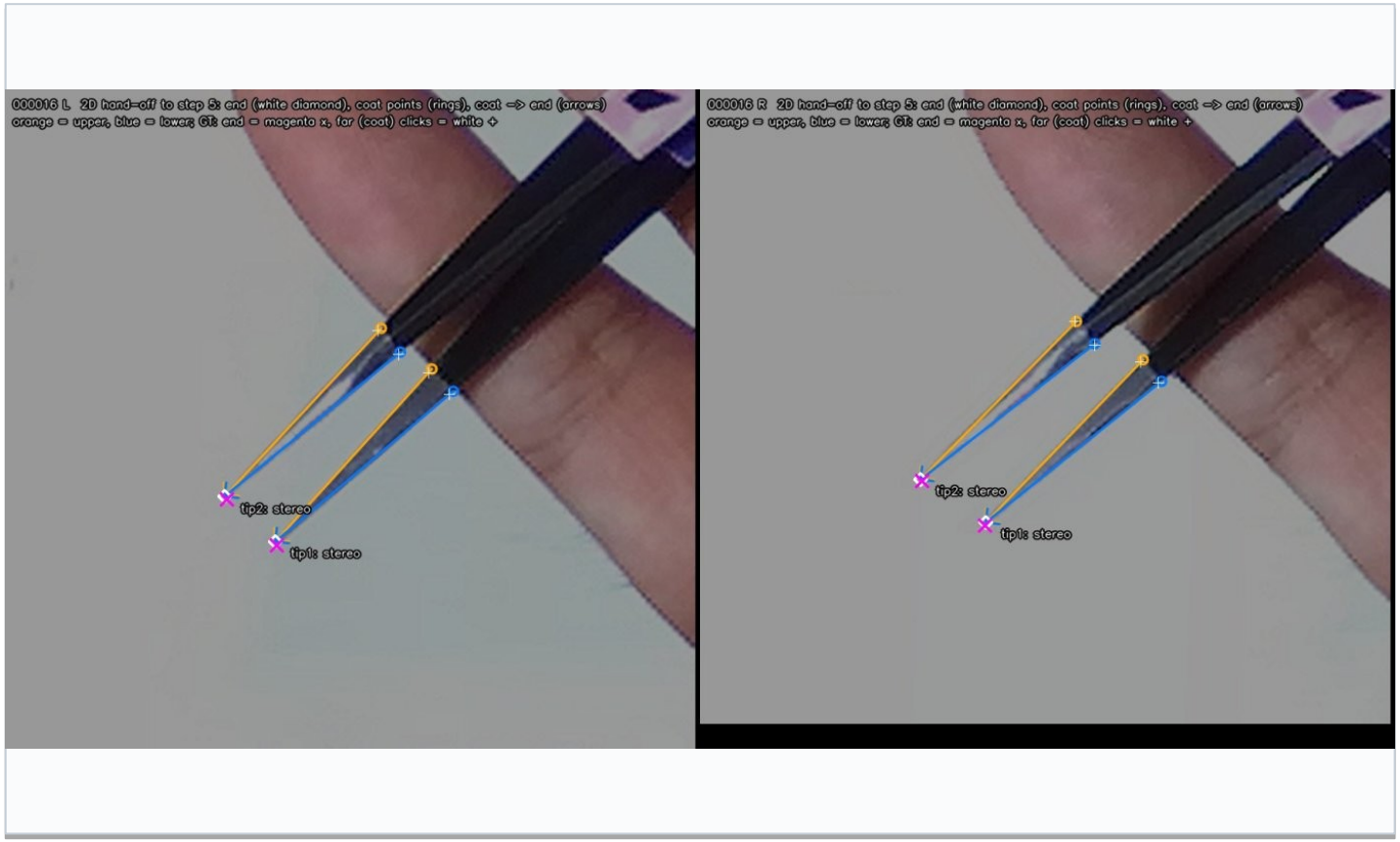


현재 남은 병목 : view 당 약 17ms 의 CPU 처리 , 특히 Python chain tracing

장비 : RTX A6000, i9-10850K | 정리된 패키지 결과 : outputs/*.json

VETUS 결과

7 stereo frames, 클릭 GT 기반 예비 평가



upper/lower label: GT 가 있는 27 건 모두 일치 · 7 개 stereo frame 모두 2D 결과 출력

흰 마름모 : endpoint · 원 : coating boundary · 자주 x/ 흰 +: 클릭 GT

끝점 endpoint 오차

검출 끝점과 사람이 클릭한 GT 사이의 Euclidean pixel distance

n = 26 view-tip
중앙값 0.97px
p90 2.07px
최대 2.48px

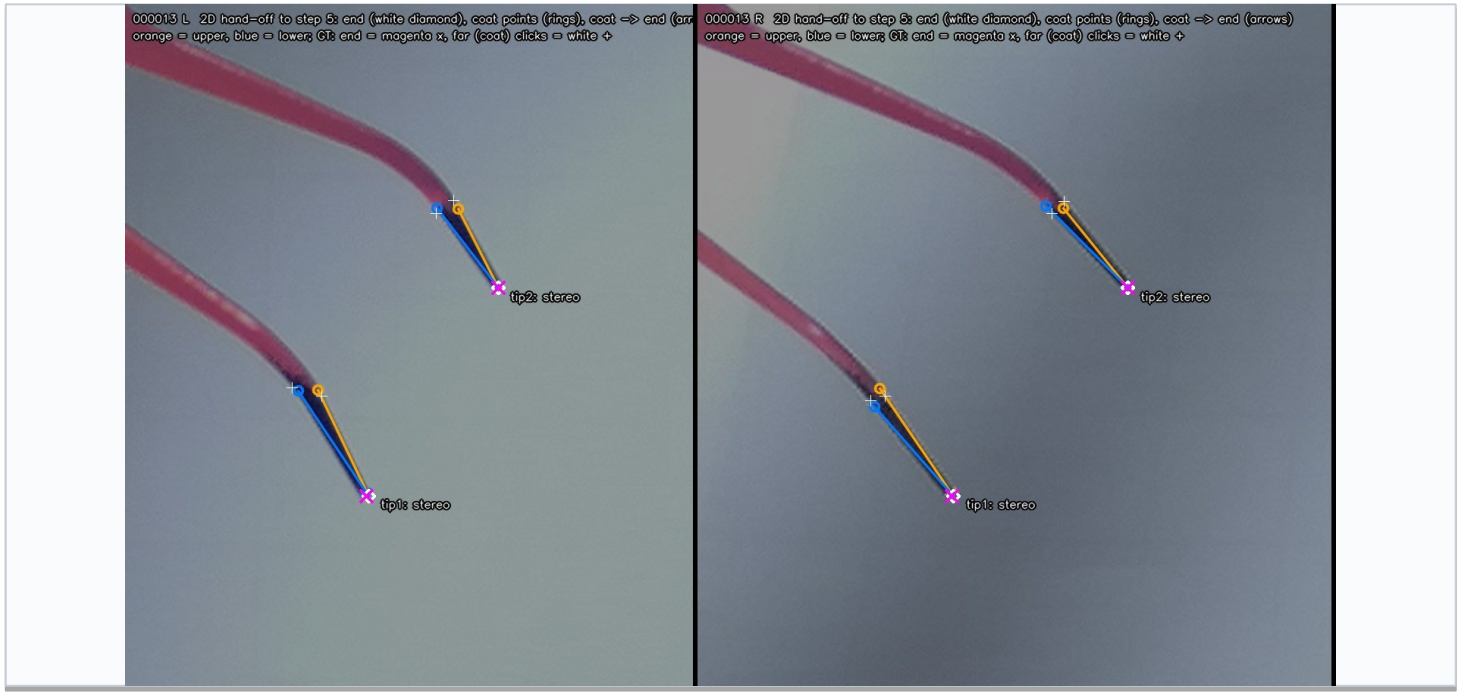
코팅 경계 오차

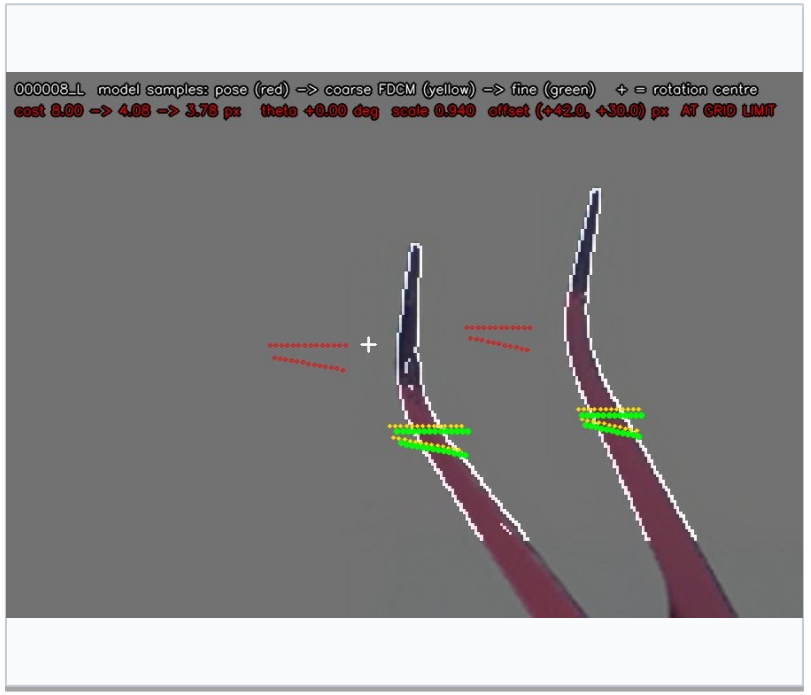
검출 coat point 와 사람이 클릭한 코팅 경계 GT 사이의 pixel distance

n = 55 points
중앙값 1.13px
p90 2.45px
최대 3.63px

Pink 결과와 pose 실패 감지

정상 pose 5 프레임은 정밀 측정 , 뒤집힌 2 프레임은 정렬 신뢰도로 거부





0.81 px
endpoint-GT 중앙값

1.40 px
endpoint-GT p90

4.06 px
endpoint-GT 최대

거부된 08·11

6D pose 의 앞뒤 / 팔 대응 오류
정렬 offset 이 탐색 grid 끝에 걸림
그대로 쓰면 endpoint 가 수십 px 벗어남

정상 pose 5 프레임 , 20 view-tip endpoint 기준

현재 상태와 한계

이번 보고 범위는 L/R 영상의 2D semantic keypoint 측정

1. Pose 뒤집힘

꺾인 pink 는 align_at_grid_limit 로 감지 가능 . 곧은 VETUS 는 upper/lower 이름만 바뀔 수 있어 pose 단계의 앞뒤 판별 필요

2. 흐린 코팅 경계

Pink 는 L/R 이 서로 다른 밝기 최대점을 잡을 수 있음 . 향후 1D brightness profile matching 이 후보

3. 현재 출력 범위

L/R endpoint, upper/lower coating point, 윤곽선 , coating→end 2D 방향과 품질 플래그를 출력

4. 규칙 기반 측정

상수 변화에 민감한 일부 팁이 존재 . 세 번째 물체로 일반화 확인 필요

5. 속도 병목

GPU 정렬은 해결 . 남은 CPU chain tracing 벡터화와 L/R view 병렬화 가능

6. 표본 규모

VETUS 7 프레임 , pink 정상 5 프레임의 예비 평가 . 더 다양한 거리 · 자세 데이터 필요

TAKEAWAY

Pose 를 keypoint 로 쓰지 않고, 영상에서 다시 측정한다

3DGS semantic 정의를 영상 edge· 밝기 ·stereo 제약과 결합

VETUS/pink 를 하나의 알고리즘과 물체별 profile 로 처리

GPU 기준 36~47ms 의 2D 팁 측정

- 클릭 GT 대비 endpoint 중앙값 : VETUS 0.97px, pink 0.81px
- 실패를 숨기지 않고 정렬 신뢰도 플래그로 검출

실행

```
python scripts/run_pose_results.py \
  --object VETUStweezers \
  --pose_run <pose result> \
  --query_dir <query> \
  --align gpu

python scripts/check_reference.py \
  outputs/VETUStweezers_gpu.json
```

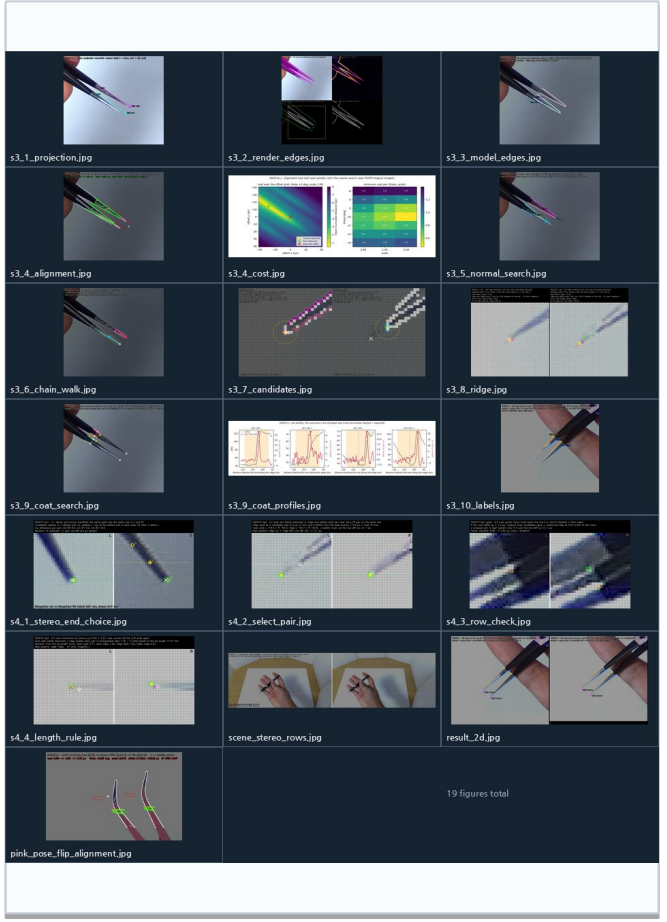
핵심 파일

pipeline.py · view.py · stereo.py
alignment.py · objects.py

Appendix | docs/figures 자료 읽는 법

README.md 와 docs/PIPELINE.md 에서 참조하는 현재 저장소의 단계별 그림

파일	단계	그림에서 확인할 내용
s3_1_projection.jpg s3_2_render_edges.jpg s3_3_model_edges.jpg s3_4_{alignment,cost}.jpg	3.1~3.4	semantic 투영 , alpha/Canny, 모델 윤곽 , 닮음 변환 정렬
s3_5_normal_search.jpg s3_6_chain_walk.jpg s3_7_candidates.jpg s3_8_ridge.jpg	3.5~3.8	법선 edge 탐색 , Canny 사슬 , 끝점 후보 , ridge 연장
s3_9_{coat_search,coat_profiles}.jpg s3_10_labels.jpg	3.9~3.10	코팅 경계 탐색 /profile 과 upper·lower 라벨
s4_1_stereo_end_choice.jpg s4_2_select_pair.jpg s4_3_row_check.jpg s4_4_length_rule.jpg	4.1~4.4	L/R 행 기반 후보 선택 , 코팅점 검사 , 수평 팁 길이 규칙
scene_stereo_rows.jpg result_2d.jpg pink_pose_flip_alignment.jpg	장면 · 결과 · 실패 사례	전체 stereo 행 , 최종 2D 출력 , pink pose 뒤집힘 사례



경로 : semantic-tip2d/docs/figures/ · 전체 19 개 JPG